Федеральное государственное автономное
образовательное учреждение высшего образования
«Национальный исследовательский университет ИТМО»

Системы искусственного интеллекта

Лабораторная работа №2

Логистическая регрессия

Предсказание выживания пассажиров «Титаника»

Выполнил:
Чэнь Хаолинь P3316 407960
Преподаватель:
Болдырева Елена Александровна

г. Санкт-Петербург, 2026

Содержание

1 Введение **2**

2 Описание метода **2**
2.1 Функция потерь и градиентный спуск 2
2.2 Метод Ньютона 3
2.3 Метрики качества 3

3 Псевдокод метода **4**

4 Результаты выполнения **4**
4.1 Описание и статистика данных 4
4.1.1 Графическое представление статистики 5
4.2 Предварительная обработка и разбиение 6
4.2.1 Преобразование признаков 6
4.2.2 Разбиение и выбор конфигурации 7
4.3 Исследование гиперпараметров 8
4.4 Оценка выбранной модели 8
4.4.1 Интерпретация коэффициентов 9

5 Примеры использования метода **10**

6 Сравнение методов **10**
6.1 Сравнительный анализ методов 10
6.2 Примеры лучшего использования каждого метода 11

7 Заключение **12**

8 Приложения **13**
8.1 Код логистической регрессии 13
8.2 Код предварительной обработки 14
8.3 Код вычисления метрик 15

1. Введение

Цель работы — реализовать логистическую регрессию с нуля и исследовать её применение к задаче предсказания выживания пассажиров «Титаника». Необходимо загрузить данные, выполнить предварительную обработку, вычислить и графически представить количество наблюдений, средние значения, стандартные отклонения, минимумы, максимумы и квантили. Затем требуется разделить данные на обучающую и тестовую выборки, реализовать сигмоиду, логарифмическую функцию потерь и градиентный спуск, исследовать влияние шага обучения, числа итераций и метода оптимизации. Для каждой комбинации параметров необходимо определить accuracy, precision, recall и F1-score и сформулировать выводы.

Реализация вычислений использует только NumPy и Pandas. Разбиение, подготовка признаков, оптимизация и вычисление метрик реализованы самостоятельно. Графики формируются в формате SVG средствами стандартной библиотеки Python.

2. Описание метода

Для бинарной случайной величины Y_i предполагается распределение Бернулли с вероятностью успеха p_i. Логит линейно зависит от признаков:

$$\log \frac{p_i}{1-p_i} = z_i = x_i^{\mathsf{T}} \beta, \qquad h_\beta(x_i) = p_i = \sigma(z_i) = \frac{1}{1+e^{-z_i}}.$$

Первая компонента x_i равна единице. При $p_i \geq 0{,}5$ предсказывается класс 1. Для защиты от переполнения сигмоида вычисляется как $1/(1+e^{-z})$ при $z \geq 0$ и как $e^z/(1+e^z)$ при $z < 0$.

2.1. Функция потерь и градиентный спуск

Максимизация правдоподобия Бернулли эквивалентна минимизации средней отрицательной логарифмической правдоподобности:

$$\begin{aligned} J(\beta) &= -\frac{1}{m} \sum_{i=1}^{m} \left[y_i \log p_i + (1-y_i) \log(1-p_i) \right] \\ &= \frac{1}{m} \sum_{i=1}^{m} \left[\log(1+e^{z_i}) - y_i z_i \right], \\ \nabla J(\beta) &= \frac{1}{m} X^{\mathsf{T}} (p - y). \end{aligned}$$

В программе второй вид вычисляется через `numpy.logaddexp(0, z)`, что сохраняет устойчивость при больших по модулю логитах. Штраф за величину коэффициентов не добавляется. Полный градиентный спуск использует все обучающие наблюдения на каждой итерации:

$$\beta^{(t+1)} = \beta^{(t)} - \alpha \nabla J(\beta^{(t)}).$$

Начальные коэффициенты нулевые; начальная потеря равна $\log 2 \approx 0{,}693147$. Обучение выполняет ровно T итераций, без досрочной остановки. Сохраняются начальная потеря и потери после каждого обновления.

2.2. Метод Ньютона

Гессиан и направление обновления определяются следующим образом:

$$H = \frac{1}{m} X^{\mathsf{T}} W X, \qquad W = \operatorname{diag}(p_i(1 - p_i)),$$

$$(H + \varepsilon I)d = \nabla J, \qquad \beta^{(t+1)} = \beta^{(t)} - \alpha d,$$

где $\varepsilon = 10^{-8}$. Диагональное слагаемое стабилизирует решение системы и не является штрафом в функции потерь. Обратная матрица явно не вычисляется. При $\alpha = 1$ используется полный шаг Ньютона, при меньших значениях — демпфированный. Одна итерация требует построения гессиана и решения системы размера 10×10, поэтому она дороже шага градиентного спуска.

2.3. Метрики качества

Положительный класс — выживший пассажир. TP, TN, FP и FN обозначают истинно положительные, истинно отрицательные, ложноположительные и ложноотрицательные ответы:

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}, \quad \text{precision} = \frac{TP}{TP + FP},$$

$$\text{recall} = \frac{TP}{TP + FN}, \quad F_1 = \frac{2TP}{2TP + FP + FN}.$$

При нулевом знаменателе метрика полагается равной нулю. Log loss оценивает вероятностные прогнозы, а четыре основные метрики зависят от порога классификации.

3. Псевдокод метода

Вход: таблица пассажиров, целевые метки y, метод оптимизации, шаг α, число итераций T. Выход: параметры модели и прогнозы вероятности выживания.

1. Разделить данные стратифицированно на обучающую и тестовую части; внутри обучающей выделить валидационную часть.
2. На внутренней обучающей части вычислить медианы, моды, средние и масштабы. Заполнить пропуски, стандартизировать числовые признаки, построить индикаторы категорий и добавить столбец единиц. Применить сохранённые преобразования к валидации.
3. Для каждой комбинации метода, α и T выполнить:
   a) инициализировать $\beta \leftarrow 0$;
   b) для $t = 1, \ldots, T$ вычислить $z \leftarrow X\beta$, $p \leftarrow \sigma(z)$ и $g \leftarrow X^{\mathsf{T}}(p - y)/m$;
   c) для градиентного спуска положить $d \leftarrow g$; для Ньютона вычислить $H \leftarrow X^{\mathsf{T}} \operatorname{diag}(p(1-p))X/m$ и решить $(H + 10^{-8}I)d = g$;
   d) обновить $\beta \leftarrow \beta - \alpha d$ и сохранить log loss; повторять до выполнения T обновлений;
   e) вычислить валидационный log loss и метрики по предсказаниям $\widehat{y} = [p \geq 0{,}5]$.
4. Выбрать конфигурацию с наименьшим валидационным log loss; при точном равенстве оставить первую в сетке. Зафиксировать выбор до оценки на тесте.
5. Заново оценить преобразования на всей обучающей выборке. Переобучить каждую конфигурацию на ней, вычислить на тесте accuracy, precision, recall и F1, сохранить результаты и кривые обучения.
6. Сохранить коэффициенты выбранной модели и параметры преобразования. Для новых пассажиров применять те же преобразования и вычислять $\sigma(X\beta)$.

4. Результаты выполнения

4.1. Описание и статистика данных

Набор содержит 891 наблюдение и 12 столбцов. Целевая переменная `Survived` принимает значения 1 (пассажир выжил) и 0 (не выжил). Выживших 342 (38,38%), погибших 549 (61,62%). Полных дубликатов строк нет; идентификаторы пассажиров уникальны.

Таблица 1: Назначение полей и выбор входных признаков

Поле	Содержание и использование
`Survived`	Бинарный целевой признак; не включается в матрицу входов.
`Age`, `Fare`	Возраст в годах и стоимость билета; числовые признаки.
`SibSp`, `Parch`	Число супругов/братьев/сестёр и родителей/детей на борту.
`Sex`, `Pclass`	Пол и класс билета; категориальные признаки.
`Embarked`	Порт посадки: C, Q или S; категориальный признак.
`PassengerId`	Идентификатор; используется только для сопоставления строк.
`Name`, `Ticket`	Имя и номер билета; исключены из базовой модели.
`Cabin`	Номер каюты; исключён из-за высокой доли пропусков.

Таблица 2: Описательная статистика до заполнения пропусков

Признак	n	Среднее	s	Min	Q_1	Q_2	Q_3	Max
`PassengerId`	891	446,00	257,35	1,00	223,50	446,00	668,50	891,00
`Survived`	891	0,38	0,49	0,00	0,00	0,00	1,00	1,00
`Pclass`	891	2,31	0,84	1,00	2,00	3,00	3,00	3,00
`Age`	714	29,70	14,53	0,42	20,12	28,00	38,00	80,00
`SibSp`	891	0,52	1,10	0,00	0,00	0,00	1,00	8,00
`Parch`	891	0,38	0,81	0,00	0,00	0,00	0,00	6,00
`Fare`	891	32,20	49,69	0,00	7,91	14,45	31,00	512,33

Здесь n — число непустых значений, s — выборочное стандартное отклонение с делителем $n-1$, Q_1, Q_2, Q_3 — квантили уровней 0,25; 0,50; 0,75 с линейной интерполяцией. Статистика идентификатора приведена для полноты, но не имеет содержательного смысла для прогнозирования.

4.1.1. Графическое представление статистики

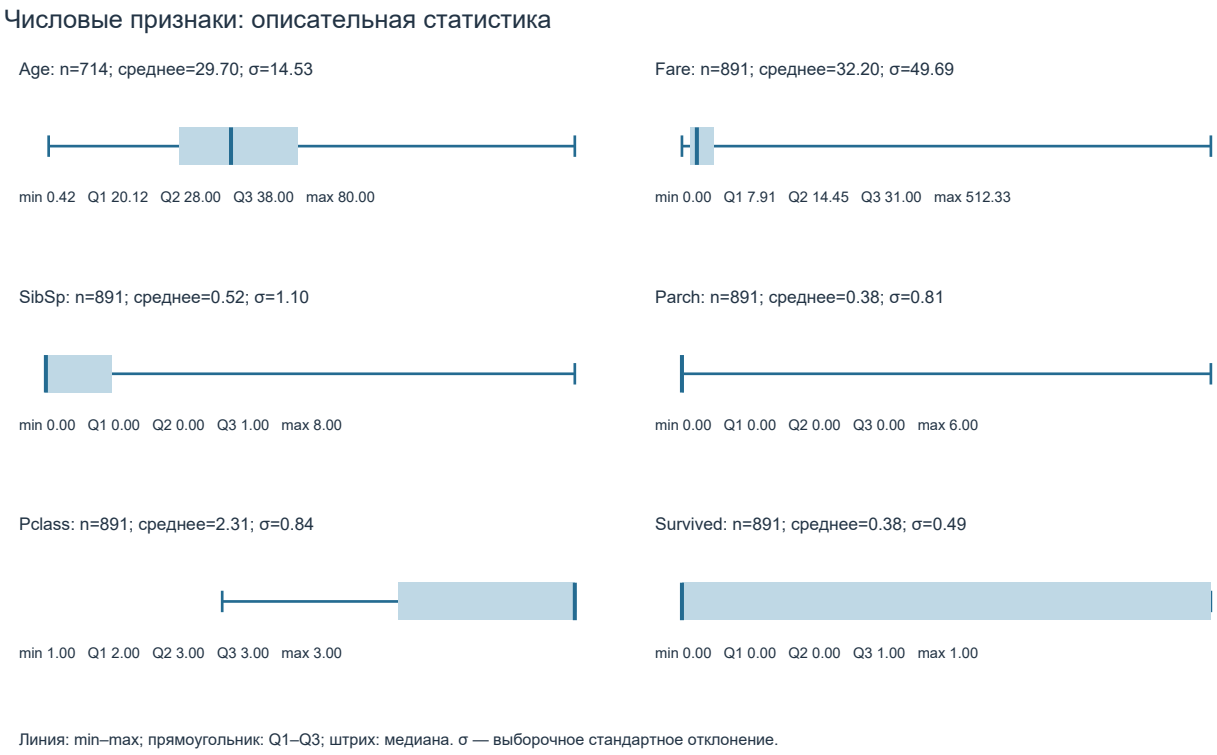


Рис. 1: Количество, среднее, стандартное отклонение и пять характеристик положения. Масштабы панелей различаются.

Средний возраст равен 29,70 года, медианный — 28 лет. Стоимость билета имеет правостороннюю асимметрию: среднее 32,20 заметно выше медианы 14,45, максимум достигает 512,33. Экстремальные значения не удалялись: сами по себе они не свидетельствуют об ошибке.

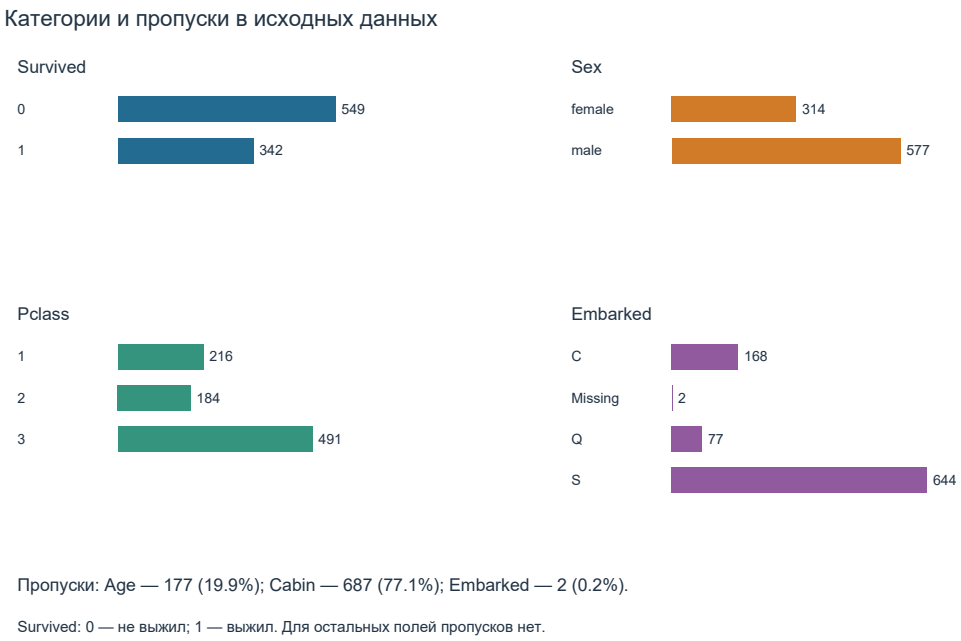


Рис. 2: Частоты категорий и пропуски. Длины столбцов пропорциональны долям от 891 наблюдения.

4.2. Предварительная обработка и разбиение

4.2.1. Преобразование признаков

Пропуски обнаружены в `Age`: 177 (19,87%), `Cabin`: 687 (77,10%) и `Embarked`: 2 (0,22%). Строки с пропусками сохраняются. Возраст заполняется медианой обучающей части, порт посадки — её модой. Для остальных числовых и категориальных признаков предусмотрены те же правила, хотя пропусков в них нет.

Четыре числовых признака `Age`, `SibSp`, `Parch`, `Fare` после заполнения пропусков стандартизируются:

$$\widetilde{x}_{ij} = \frac{x_{ij} - \mu_j}{\sigma_j}, \qquad \sigma_j = \sqrt{\frac{1}{m}\sum_{i=1}^{m}(x_{ij} - \mu_j)^2}.$$

Параметры μ_j и σ_j оцениваются только на обучающей части. Для стандартизации используется делитель m, в отличие от описательной статистики. При нулевом стандартном отклонении масштаб заменяется единицей.

Категории кодируются индикаторами с исключением базового уровня: женщина для `Sex`, первый класс для `Pclass`, порт C для `Embarked`. Получаются пять индикаторов: `Sex_male`, `Pclass_2`, `Pclass_3`, `Embarked_Q`, `Embarked_S`. Класс билета не трактуется как числовой признак с равными расстояниями между категориями. Всего используются девять признаков и свободный коэффициент, то есть десять параметров.

Таблица 3: Параметры преобразования при обучении на 713 объектах

Признак	Медиана	μ	σ
`Age`	28,0000	28,9967	12,7471
`SibSp`	0,0000	0,4979	1,0326
`Parch`	0,0000	0,3843	0,8073
`Fare`	14,4542	32,4917	51,6504

Мода порта посадки — S. Тестовые значения не участвуют в вычислении медиан, мод и параметров стандартизации.

4.2.2. Разбиение и выбор конфигурации

Использовано стратифицированное разбиение приблизительно 80:20 с `seed=42`. Обучающая выборка содержит 713 объектов, тестовая — 178. Внутри обучающей части выделена валидационная выборка, также стратифицированно, с `seed=43`.

Таблица 4: Размеры выборок и распределение классов

Выборка	Всего	Класс 0	Класс 1
Обучающая целиком	713	439	274
Внутренняя обучающая	570	351	219
Валидационная	143	88	55
Тестовая	178	110	68

Сначала каждая конфигурация обучается на 570 объектах и оценивается на 143 валидационных объектах; преобразования также оцениваются только по этим 570 объектам. Критерий выбора — минимальный валидационный log loss. При точном равенстве выбирается первая конфигурация в сетке. После фиксации выбора все конфигурации переобучаются на 713 объектах и оцениваются на одном тестовом наборе, как требует задание. Тестовые метрики не используются для повторного выбора параметров.

4.3. Исследование гиперпараметров

Для градиентного спуска проверены $\alpha \in \{0{,}001; 0{,}01; 0{,}1; 1\}$ и $T \in \{100; 1000; 5000\}$; для метода Ньютона — $\alpha \in \{0{,}1; 0{,}5; 1\}$ и $T \in \{3; 10; 30\}$. Всего рассмотрена 21 комбинация. Разные диапазоны итераций отражают различную скорость сходимости. Время работы не измерялось, поэтому число итераций не трактуется как прямое сравнение времени.

Таблица 5: Градиентный спуск: валидационная потеря и тестовые метрики

α	T	J_v	J_t	A	P	R	F_1
0,001	100	0,6827	0,6830	0,6461	0,7778	0,1029	0,1818
0,001	1000	0,6207	0,6253	0,6517	0,8000	0,1176	0,2051
0,001	5000	0,5375	0,5594	0,6854	0,8000	0,2353	0,3636
0,01	100	0,6206	0,6252	0,6517	0,8000	0,1176	0,2051
0,01	1000	0,4980	0,5295	0,7247	0,8065	0,3676	0,5051
0,01	5000	0,4278	0,4587	0,8146	0,8182	0,6618	0,7317
0,1	100	0,4978	0,5294	0,7247	0,8065	0,3676	0,5051
0,1	1000	0,4174	0,4410	0,8146	0,7966	0,6912	0,7402
0,1	5000	0,4066	0,4235	0,8034	0,7705	0,6912	0,7287
1	100	0,4173	0,4409	0,8146	0,7966	0,6912	0,7402
1	1000	0,4056	0,4228	0,7921	0,7541	0,6765	0,7132
1	5000	0,4055	0,4228	0,7921	0,7541	0,6765	0,7132

Таблица 6: Метод Ньютона: валидационная потеря и тестовые метрики

α	T	J_v	J_t	A	P	R	F_1
0,1	3	0,5860	0,5854	0,7978	0,7500	0,7059	0,7273
0,1	10	0,4728	0,4757	0,8034	0,7619	0,7059	0,7328
0,1	30	0,4108	0,4249	0,8034	0,7705	0,6912	0,7287
0,5	3	0,4297	0,4378	0,8090	0,7656	0,7206	0,7424
0,5	10	0,4057	0,4228	0,7921	0,7541	0,6765	0,7132
0,5	30	0,4055	0,4228	0,7921	0,7541	0,6765	0,7132
1	3	0,4056	0,4227	0,7921	0,7541	0,6765	0,7132
1	10*	0,4055	0,4228	0,7921	0,7541	0,6765	0,7132
1	30	0,4055	0,4228	0,7921	0,7541	0,6765	0,7132

Здесь J_v — потеря на валидации при внутреннем обучении, J_t — потеря на тесте после переобучения. A, P, R обозначают accuracy, precision и recall. Звёздочкой отмечена выбранная конфигурация. Все четыре метрики приведены для каждой комбинации; округление выполнено только при отображении.

Выбран метод Ньютона с $\alpha = 1$ и $T = 10$: $J_v = 0{,}4055198454$. Результат при 30 итерациях совпадает, поэтому остаётся более ранняя конфигурация сетки. Градиентный спуск с $\alpha = 1$, $T = 5000$ даёт практически то же значение; отличие порядка 10^{-14} не имеет содержательного значения. Выбранный метод представляет группу сошедшихся решений, а не статистически доказанного единственного победителя.

4.4. Оценка выбранной модели

После обучения на 713 объектах выбранная модель получила на 178 тестовых объектах:

$$\text{accuracy} = 0{,}7921, \qquad \text{precision} = 0{,}7541, \qquad \text{recall} = 0{,}6765, \qquad F_1 = 0{,}7132.$$

Обучающая потеря равна 0,447637, тестовая — 0,422788. Константный классификатор, всегда предсказывающий класс 0, имеет accuracy 0,6180 и нулевые precision, recall и F1. Accuracy выбранной модели выше базовой на 17,42 процентного пункта.

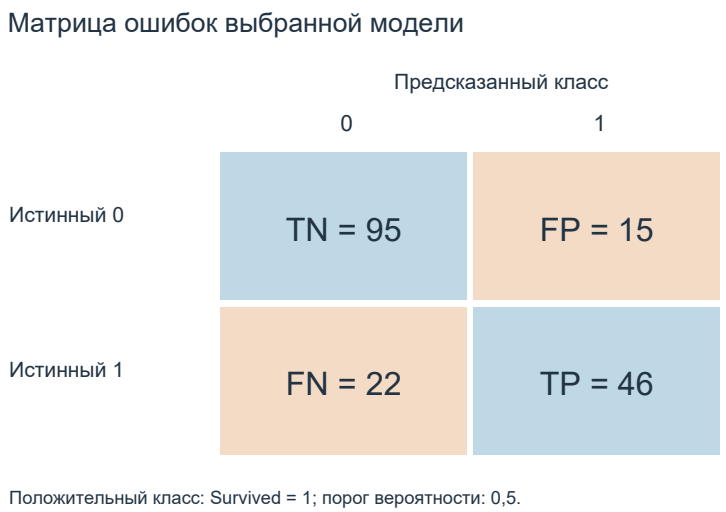


Рис. 3: Матрица ошибок на тесте: метод Ньютона, $\alpha = 1$, $T = 10$.

Верно распознаны 95 из 110 погибших и 46 из 68 выживших. Получены 15 ложноположительных и 22 ложноотрицательных ответа. При фиксированном пороге модель пропускает примерно треть выживших.

4.4.1. Интерпретация коэффициентов

Таблица 7: Коэффициенты и условные отношения шансов выбранной модели

Параметр	β_j	e^{β_j}
`Intercept`	2,7425	15,5258
`Age`	-0,5812	0,5592
`SibSp`	-0,4432	0,6419
`Parch`	0,0149	1,0150
`Fare`	0,1465	1,1578
`Sex_male`	-2,6356	0,0717
`Pclass_2`	-0,9629	0,3818
`Pclass_3`	-1,9884	0,1369
`Embarked_Q`	-0,1879	0,8287
`Embarked_S`	-0,4642	0,6286

Для числового признака e^{β_j} соответствует увеличению на одно обучающее стандартное отклонение при остальных неизменных признаках. Для индикатора сравнивается категория с базовой: например, отношение шансов для мужчины относительно женщины равно 0,0717, а для третьего класса относительно первого — 0,1369. Это условные ассоциации, а не причинные эффекты. Свободный коэффициент соответствует базовым категориям при нулевых стандартизированных признаках.

5. Примеры использования метода

В рассматриваемой задаче модель сопоставляет сведения о пассажире с вероятностью выживания. Например, она позволяет оценить условную связь пола, возраста и класса билета с исходом и получить интерпретируемую базовую классификацию. При этом коэффициенты описывают статистические зависимости, а не доказывают причинность.

Логистическая регрессия также применима к классификации сообщений как нежелательных или обычных, если текст заранее представлен числовыми признаками. Вероятностный ответ позволяет менять порог с учётом цены ошибочной блокировки, а линейные коэффициенты помогают интерпретировать вклад признаков.

Ещё один пример — прогноз отклика клиента на предложение по истории взаимодействий. Метод подходит как простая проверяемая базовая модель для бинарного отклика. Он особенно удобен при умеренном числе признаков, необходимости объяснить прогноз и ограниченных вычислительных ресурсах; сложные нелинейные зависимости могут потребовать расширения признаков или другой модели.

6. Сравнение методов

6.1. Сравнительный анализ методов

Сравниваются два способа обучения одной логистической модели: полный градиентный спуск и метод Ньютона. Набор признаков, функция потерь, начальные коэффициенты и порог одинаковы; различаются правило обновления, шаг и число итераций.

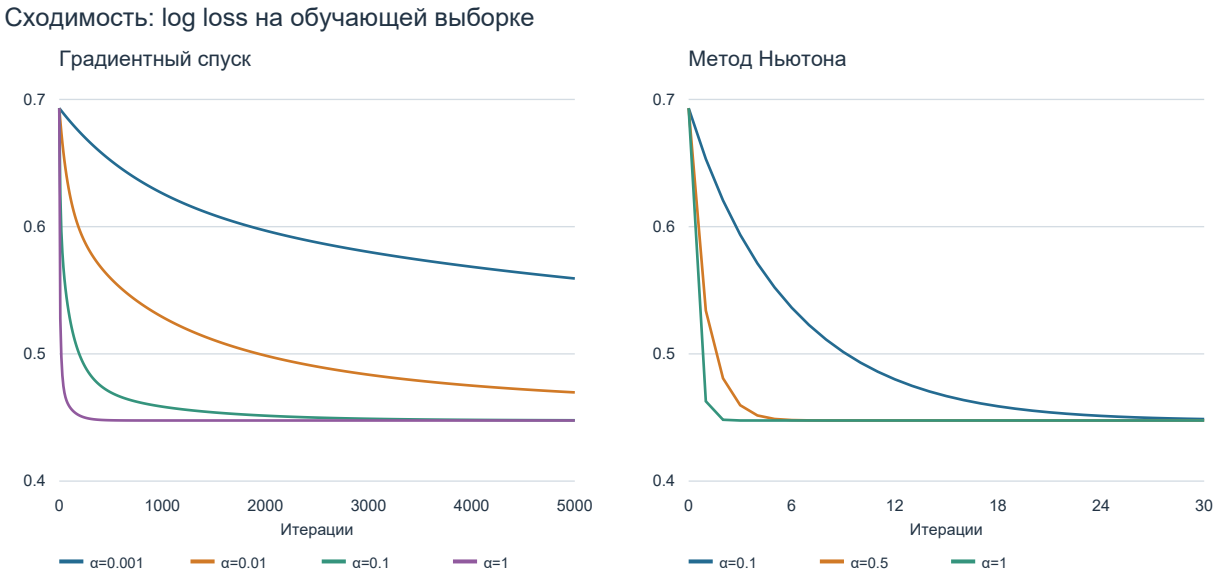


Рис. 4: Потери при окончательном обучении. Показаны конфигурации с максимальным числом итераций для соответствующего метода.

При $\alpha = 0{,}001$ градиентный спуск обучается медленно: увеличение числа итераций со 100 до 5000 снижает обучающую потерю с 0,683164 до 0,559256, но тестовая полнота остаётся низкой — 0,2353. Высокая precision при малой полноте связана с небольшим числом предсказаний класса 1 и не означает высокого качества в целом.

При шагах 0,1 и 1 сходимость заметно быстрее. Градиентный спуск с шагом 1 за 5000 итераций достигает той же обучающей потери 0,447637, что и метод Ньютона с шагом 1 за 10 итераций. Для проверенных значений расходимости нет; это не гарантирует устойчивости при любом большем шаге.

Демпфирование замедляет метод Ньютона: при $\alpha = 0{,}1$ после 30 итераций обучающая потеря равна 0,448750, а при $\alpha = 0{,}5$ уже после 10 итераций — 0,447639. При полном шаге трёх итераций достаточно для близкого к оптимуму решения, а десяти — для выхода на плато.

Минимизация log loss не обязана максимизировать F1 при фиксированном пороге. На тесте наибольшая accuracy, 0,8146, наблюдается у градиентного спуска при $(\alpha, T) = (0{,}01, 5000)$, $(0{,}1, 1000)$ и $(1, 100)$. Наибольшая F1, 0,7424, получена методом Ньютона при $(0{,}5, 3)$; для него accuracy равна 0,8090, precision — 0,7656, recall — 0,7206. Это описание проведённого сравнения, а не основание менять заранее установленный критерий выбора.

Сходимость потерь сопровождается изменениями вероятностей около порога 0,5. Поэтому увеличение числа итераций может уменьшать log loss и одновременно ухудшать пороговые метрики. Например, при шаге градиентного спуска 0,1 переход от 1000 к 5000 итерациям уменьшает тестовую потерю с 0,441004 до 0,423511, но снижает F1 с 0,7402 до 0,7287.

На валидации максимальную F1, 0,7810, даёт градиентный спуск с $\alpha = 0{,}01$, $T = 5000$. Если цель состоит именно в максимизации F1, это обоснованная альтернатива по валидационному критерию. В настоящем эксперименте критерием остаётся log loss, поскольку он оценивает вероятностную модель и связан с функцией обучения.

Оценки относятся к одному фиксированному разбиению. Различия в несколько правильных ответов на 178 объектах не доказывают устойчивого превосходства метода. Доверительные интервалы и повторная перекрёстная проверка не рассчитывались. Модель ограничена линейностью логита, простым заполнением возраста и отсутствием взаимодействий признаков; имя, билет и каюта не использованы. При случайном разбиении связанные пассажиры одной семьи могут попасть в разные части, что дополнительно ограничивает интерпретацию переносимости результата.

Таблица 8: Свойства методов оптимизации; m — число объектов, d — число параметров

Характеристика	Градиентный спуск	Метод Ньютона
Информация о функции	Только градиент	Градиент и гессиан
Стоимость итерации	Порядка $O(md)$	Порядка $O(md^2 + d^3)$
Дополнительная память	$O(m + d)$ в реализации	$O(md + d^2)$ в реализации
Преимущество	Простота и дешёвое обновление	Быстрая локальная сходимость
Ограничение	Чувствительность к шагу и масштабу	Более дорогая итерация; возможная плохая обусловленность гессиана
Итог эксперимента	$\alpha = 1$, $T = 5000$: $J_{train} = 0{,}447637$	$\alpha = 1$, $T = 10$: $J_{train} = 0{,}447637$

Оценки дополнительной памяти не включают хранение исходной матрицы признаков. Векторизованная реализация создаёт временные массивы размера $O(m)$ и, при построении гессиана, $O(md)$. Матрица W размера $m \times m$ явно не формируется.

6.2. Примеры лучшего использования каждого метода

Градиентный спуск предпочтителен при большом числе признаков, когда хранение и решение системы с гессианом слишком дороги. Например, для модели фильтрации сообщений с тысячами текстовых признаков его линейная зависимость стоимости шага от d делает метод более практичным. Для ускорения требуется подобрать шаг и масштабировать признаки.

Метод Ньютона удобен для задач с небольшим числом параметров и хорошо обусловленным гессианом, когда требуется быстро приблизиться к минимуму потерь. Titanic с десятью пара-

метрами относится к этому случаю: система мала, и полный шаг быстро приводит к плато. При почти линейно зависимых признаках или разделимости классов нужны дополнительные меры стабилизации; использованное малое диагональное слагаемое не заменяет полноценную регуляризацию.

7. Заключение

Выполнена бинарная классификация пассажиров «Титаника» по девяти подготовленным признакам. Исследованы числовые и категориальные характеристики 891 наблюдения, визуализированы статистики и пропуски, выполнено стратифицированное разбиение. Заполнение пропусков и стандартизация рассчитаны только по соответствующим обучающим частям.

С нуля реализованы устойчивая сигмоида, log loss, полный градиентный спуск и демпфированный метод Ньютона. Для всех 21 комбинаций вычислены accuracy, precision, recall и F1. По минимальному валидационному log loss выбран метод Ньютона с шагом 1 и 10 итерациями; тестовые accuracy и F1 составили 0,7921 и 0,7132 соответственно.

Малый шаг градиентного спуска приводит к недообучению в пределах заданного числа итераций. Для приближения к минимуму log loss в исследованной сетке подходят шаг 1 и 5000 итераций градиентного спуска либо шаг 1 и 10 итераций Ньютона. Метод Ньютона требует меньше итераций, но каждая из них сложнее. После достижения плато дальнейшее обучение почти не меняет результат. Лучшие параметры зависят от критерия: минимум вероятностной потери может не соответствовать максимальной F1 при пороге 0,5.

8. Приложения

8.1. Код логистической регрессии

Ниже приведён вычислительный код модели, подготовки данных и метрик. Модули запуска эксперимента, построения графиков и применения сохранённой модели входят в программный проект `src/titanic`.

```
"""Numerically stable binary logistic regression implemented with NumPy."""

import numpy as np


def sigmoid(logits):
    """Evaluate the sigmoid without exponential overflow."""
    logits = np.asarray(logits, dtype=float)
    values = np.empty_like(logits)
    positive = logits >= 0
    values[positive] = 1.0 / (1.0 + np.exp(-logits[positive]))
    exponent = np.exp(logits[~positive])
    values[~positive] = exponent / (1.0 + exponent)
    return values


def log_loss(target, logits):
    """Mean Bernoulli negative log likelihood, evaluated from logits."""
    return float(np.mean(np.logaddexp(0.0, logits) - target * logits))


def derivatives(matrix, target, weights):
    probabilities = sigmoid(matrix @ weights)
    gradient = matrix.T @ (probabilities - target) / len(target)
    curvature = probabilities * (1.0 - probabilities)
    hessian = matrix.T @ (curvature[:, None] * matrix) / len(target)
    return gradient, hessian


class LogisticRegression:
    """Full-batch gradient descent or damped Newton with fixed step size."""

    def __init__(self, method='gd', learning_rate=0.1, iterations=1000):
        if method not in {'gd', 'newton'}:
            raise ValueError('method must be 'gd' or 'newton'')
        if not np.isfinite(learning_rate) or learning_rate <= 0:
            raise ValueError('learning_rate must be finite and positive')
        if not isinstance(iterations, int) or iterations < 1:
            raise ValueError('iterations must be a positive integer')
        self.method = method
        self.learning_rate = learning_rate
        self.iterations = iterations

    def fit(self, matrix, target):
        matrix = np.asarray(matrix, dtype=float)
        target = np.asarray(target, dtype=float)
        if matrix.ndim != 2 or target.shape != (len(matrix),) or not len(target):
            raise ValueError('Expected a nonempty matrix and matching target')
        if not np.isfinite(matrix).all() or not np.isin(target, [0, 1]).all():
            raise ValueError('Inputs must be finite; target must contain 0 or 1')
        self.weights = np.zeros(matrix.shape[1])
        self.loss_history = [log_loss(target, matrix @ self.weights)]
        for _ in range(self.iterations):
            probabilities = sigmoid(matrix @ self.weights)
            gradient = matrix.T @ (probabilities - target) / len(target)
            if self.method == 'newton':
                curvature = probabilities * (1.0 - probabilities)
                hessian = matrix.T @ (curvature[:, None] * matrix) / len(target)
                # Stabilize the linear solve; the objective has no L2 penalty.
                direction = np.linalg.solve(
                    hessian + 1e-8 * np.eye(matrix.shape[1]), gradient
                )
            else:
                direction = gradient
            self.weights -= self.learning_rate * direction
            loss = log_loss(target, matrix @ self.weights)
            if not np.isfinite(loss):
```

```
                raise FloatingPointError('Training produced a non-finite loss')
            self.loss_history.append(loss)
        return self

    def predict_proba(self, matrix):
        return sigmoid(np.asarray(matrix, dtype=float) @ self.weights)

    def predict(self, matrix, threshold=0.5):
        return (self.predict_proba(matrix) >= threshold).astype(int)
```

8.2. Код предварительной обработки

```
"""Training-only imputation, fixed reference coding, and stratified splitting."""

import numpy as np
import pandas as pd

NUMERIC = ['Age', 'SibSp', 'Parch', 'Fare']
CATEGORIES = {
    'Sex': ['female', 'male'],
    'Pclass': [1, 2, 3],
    'Embarked': ['C', 'Q', 'S'],
}
FEATURES = NUMERIC + ['Sex_male', 'Pclass_2', 'Pclass_3', 'Embarked_Q', 'Embarked_S']


def stratified_split(target, fraction=0.2, seed=42):
    """Return disjoint positional indices, retaining both classes in each part."""
    target = np.asarray(target)
    if not 0 < fraction < 1:
        raise ValueError('fraction must lie strictly between zero and one')
    rng = np.random.default_rng(seed)
    train, test = [], []
    for label in np.unique(target):
        indices = rng.permutation(np.flatnonzero(target == label))
        count = round(len(indices) * fraction)
        if count < 1 or count >= len(indices):
            raise ValueError('Each class needs observations in both splits')
        test.extend(indices[:count])
        train.extend(indices[count:])
    return rng.permutation(train), rng.permutation(test)


class Preprocessor:
    """Impute numeric medians and categorical modes; scale numeric columns only."""

    @staticmethod
    def validate(frame):
        missing = set(NUMERIC + list(CATEGORIES)) - set(frame.columns)
        if missing:
            raise ValueError(f'Missing columns: {sorted(missing)}')
        numeric = frame[NUMERIC].astype(float)
        if np.isinf(numeric.to_numpy()).any() or (numeric < 0).any().any():
            raise ValueError('Numeric features must be nonnegative and not infinite')
        for name, allowed in CATEGORIES.items():
            if not frame[name].dropna().isin(allowed).all():
                raise ValueError(f'Unsupported category in {name}; expected {allowed}')

    def fit(self, frame):
        self.validate(frame)
        self.medians = frame[NUMERIC].astype(float).median()
        if self.medians.isna().any():
            raise ValueError('A numeric feature is entirely missing')
        self.modes = {}
        for name in CATEGORIES:
            mode = frame[name].mode()
            if mode.empty:
                raise ValueError(f'Categorical feature {name} is entirely missing')
            value = mode.iloc[0]
            self.modes[name] = value.item() if hasattr(value, 'item') else value
        filled = frame[NUMERIC].astype(float).fillna(self.medians)
        self.means = filled.mean()
        self.scales = filled.std(ddof=0).replace(0.0, 1.0)
        return self
```

```
63
64      def transform(self, frame):
65          self.validate(frame)
66          numeric = frame[NUMERIC].astype(float).fillna(self.medians)
67          blocks = [
68              np.ones((len(frame), 1)),
69              ((numeric - self.means) / self.scales).to_numpy(),
70          ]
71          for name, allowed in CATEGORIES.items():
72              values = frame[name].fillna(self.modes[name])
73              blocks.extend(
74                  (values == category).to_numpy(dtype=float)[:, None]
75                  for category in allowed[1:]
76              )
77          matrix = np.column_stack(blocks)
78          if not np.isfinite(matrix).all():
79              raise ValueError('Preprocessing produced non-finite features')
80          return matrix
81
82      def to_dict(self):
83          return {
84              'medians': self.medians.to_dict(),
85              'modes': self.modes,
86              'means': self.means.to_dict(),
87              'scales': self.scales.to_dict(),
88              'features': FEATURES,
89          }
90
91      @classmethod
92      def from_dict(cls, state):
93          if state['features'] != FEATURES:
94              raise ValueError('Model feature schema does not match this implementation')
95          result = cls()
96          for name in ['medians', 'means', 'scales']:
97              setattr(result, name, pd.Series(state[name]).reindex(NUMERIC))
98          result.modes = state['modes']
99          return result
```

8.3. Код вычисления метрик

```
"""Binary metrics with class 1 (survived) as the positive class."""

import numpy as np


def classification_metrics(target, predicted):
    target, predicted = np.asarray(target), np.asarray(predicted)
    if target.ndim != 1 or target.shape != predicted.shape or target.size == 0:
        raise ValueError('Expected matching nonempty one-dimensional arrays')
    if not np.isin(target, [0, 1]).all() or not np.isin(predicted, [0, 1]).all():
        raise ValueError('Labels must be binary')
    tn = int(np.sum((target == 0) & (predicted == 0)))
    fp = int(np.sum((target == 0) & (predicted == 1)))
    fn = int(np.sum((target == 1) & (predicted == 0)))
    tp = int(np.sum((target == 1) & (predicted == 1)))
    return {
        'accuracy': (tn + tp) / len(target),
        'precision': tp / (tp + fp) if tp + fp else 0.0,
        'recall': tp / (tp + fn) if tp + fn else 0.0,
        'f1': 2 * tp / (2 * tp + fp + fn) if 2 * tp + fp + fn else 0.0,
        'tn': tn,
        'fp': fp,
        'fn': fn,
        'tp': tp,
    }
```